TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Macroarea di Ingegneria
Dipartimento di Ingegneria Civile e Ingegneria Informatica

Communication in Distributed Systems
Message Oriented Middleware

Corso di Sistemi Distribuiti e Cloud Computing
A.A. 2026/27

Valeria Cardellini

Laurea Magistrale in Ingegneria Informatica

Message-oriented communication

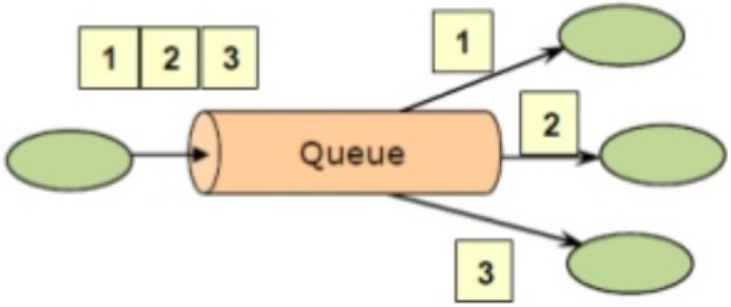
- RPC provides distribution transparency over socket programming
- But still coupling between interacting entities
  - Temporal: caller waits for a reply
  - Spatial: caller must know callee's identity or address
  - Functionality and communication are tightly coupled
- How to improve decoupling and flexibility?
- Message-oriented communication
  - Transient
    - Berkeley sockets, Message Passing Interface (MPI)
  - Persistent
    - **Message Oriented Middleware (MOM)**

Message-oriented middleware

- Communication middleware that supports sending and receiving messages in a persistent way
  - Provides intermediate-term storage for messages
- Loose coupling among system/app components
  - Supports temporal and spatial decoupling
  - Can also support synchronization decoupling
- Goals: increase performance, scalability, and reliability
  - Commonly used in serverless and microservice architectures
- Communication patterns:
  - **Message queue**
  - **Publish-subscribe** (pub/sub)
- Related systems:
  - **Message queue system** (MQS)
  - **Pub/sub system**

Message queue pattern

- Messages sent to a queue are stored until retrieved by a consumer
- Multiple producers can send messages to the queue
- Multiple consumers can receive messages from the queue
- Communication is one-to-one: each message is delivered to a single consumer



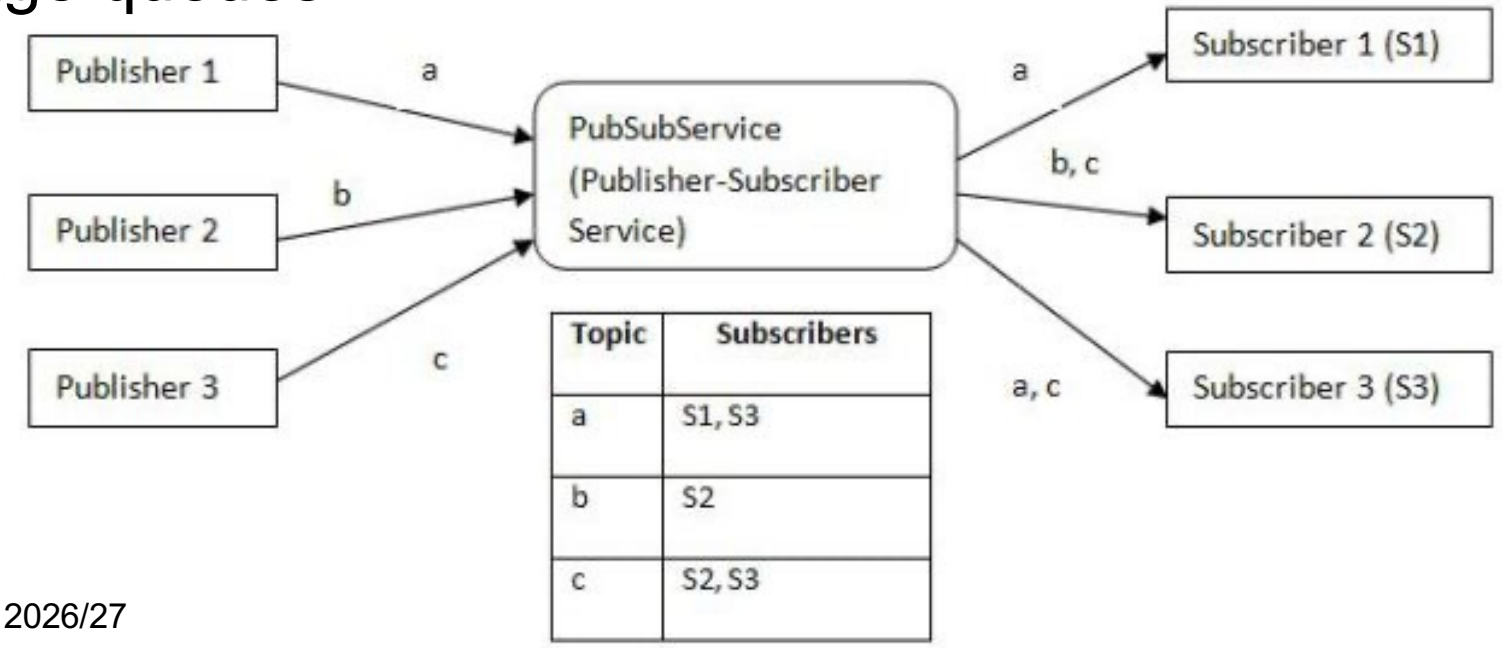
- Use cases:
  - Task scheduling, load balancing, logging or tracing

Message queue API

- Typical calls in MQS:
  - **`put`**: non-blocking send
    - Inserts a message into the queue immediately
  - **`get`**: blocking receive
    - Waits until the queue is non-empty and retrieves a message
    - Variant: can search for a specific message in the queue
  - **`poll`**: non-blocking receive
    - Checks the queue and retrieves a message if available
    - Never blocks
  - **`notify`**: non-blocking receive
    - Installs a callback handler that is automatically invoked when a message arrives in the queue

Publish/subscribe pattern

- Producers (publishers) send asynchronous messages, such as event notifications
- Consumers (subscribers) express interest in specific topics by subscribing
- Decoupling in time and space: publishers and subscribers do not need to interact directly
- Supports one-to-many communication, unlike message queues



Publish/subscribe pattern

- Multiple consumers can subscribe to a topic, with or without filters
- An *event dispatcher* collects subscriptions and routes messages to all matching subscribers
  - For scalability, the dispatcher is typically distributed
- High degree of decoupling among components
  - Components can be easily added or removed: ideal for dynamic environments
- Use cases:
  - Event notification, multicasting
- Comparison with message queue

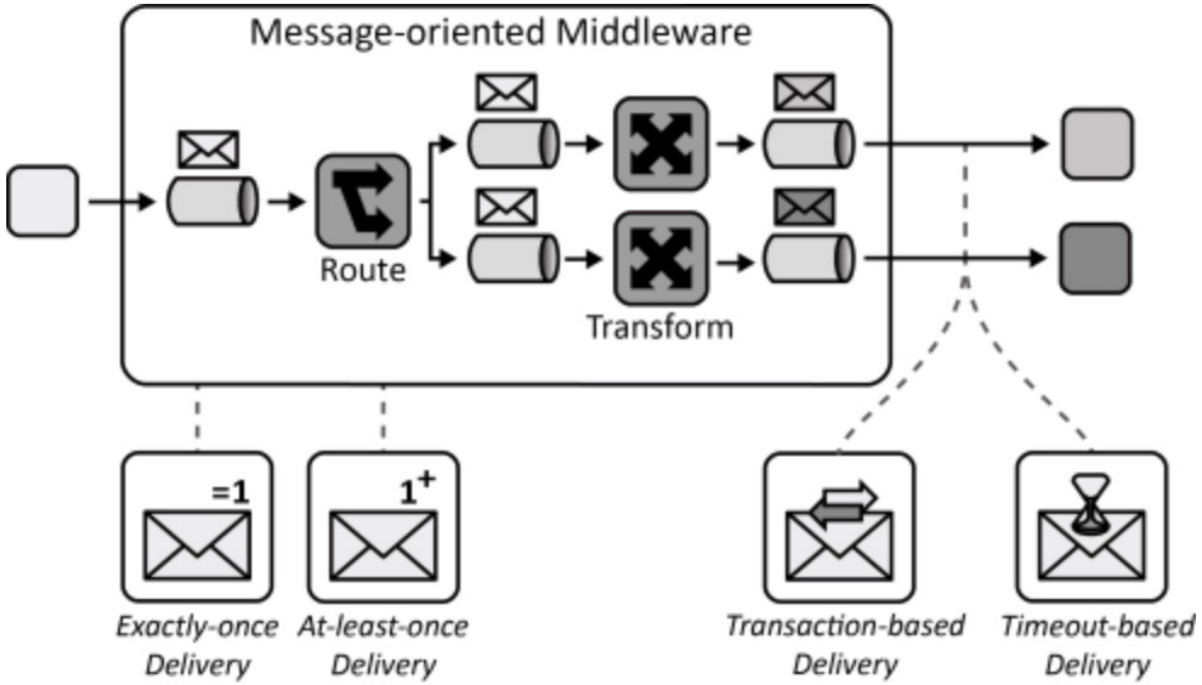
Feature	Message queue	Pub/sub
Delivery	One-to-one	One-to-many
Consumers	Single consumer per message	Multiple consumers per message

Publish/subscribe API

- Typical calls in a pub/sub system:
  - `publish(event)`: called by a publisher to publish an event
    - Events can be any data type and may include meta-data
  - `subscribe(filter_expr, notify_cb, expiry) → sub_handle`: called by a subscriber to subscribe to events
    - Inputs: filter expression, reference to a notification callback, and subscription expiry time
    - Returns a subscription handle
  - `notify_cb(sub_handle, event)`: called by pub/sub system to deliver to a matching event to subscribers
  - `unsubscribe(sub_handle)`: called by the subscriber to remove the subscription

MOM functionalities

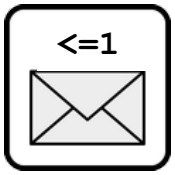
- MOM simplifies distributed communication by handling:
  - Availability of components: delivery semantics and models
  - Addressing and routing: locating the recipient(s) for messages and delivering messages along appropriate paths: message routing
  - Message format transformations: message broker



https://www.cloudcomputingpatterns.org/message_oriented_middleware

MOM delivery semantics

At-most-once delivery

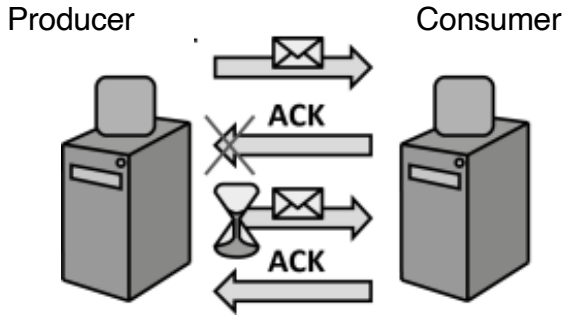


- Each message is delivered no more than once
- Messages may be lost, but are never redelivered

At-least-once delivery



- Messages are never lost, but they may be delivered multiple times
  - Since messages can be duplicated, the consumer-side application must be able to handle duplicate processing (or be sure that re-processing the same message does not cause errors)
- Mechanism*:*
  - Consumer sends ack for each message; MOM resends message if ack is missing

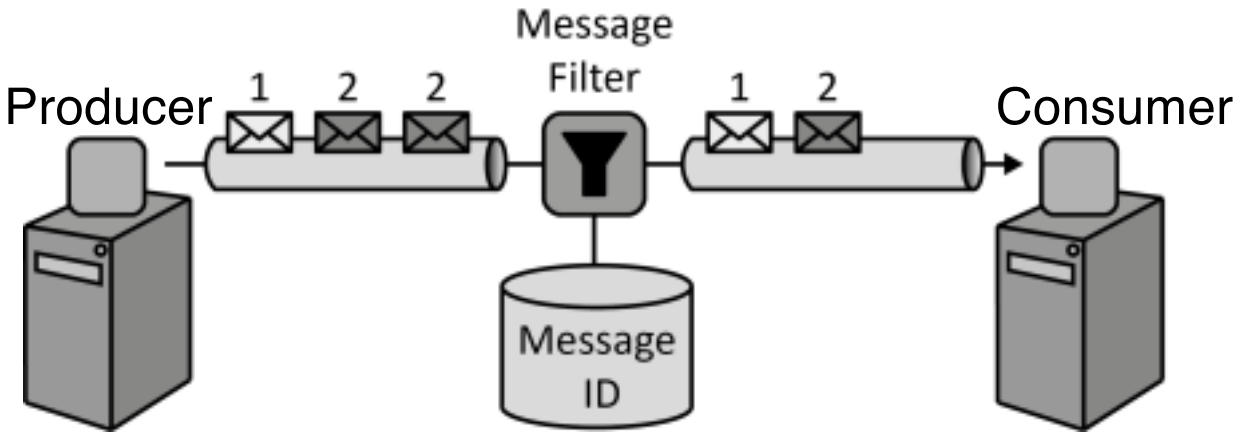


MOM delivery semantics

Exactly-once delivery



- Message is delivered exactly once to consumer
- Mechanisms
  - Deduplication logic: MOM filters message duplicates
    - Each message has a unique ID
  - Idempotent consumers: can safely process the same message multiple times
  - Messages survive MOM failures
- Note: what systems provide is exactly-once processing

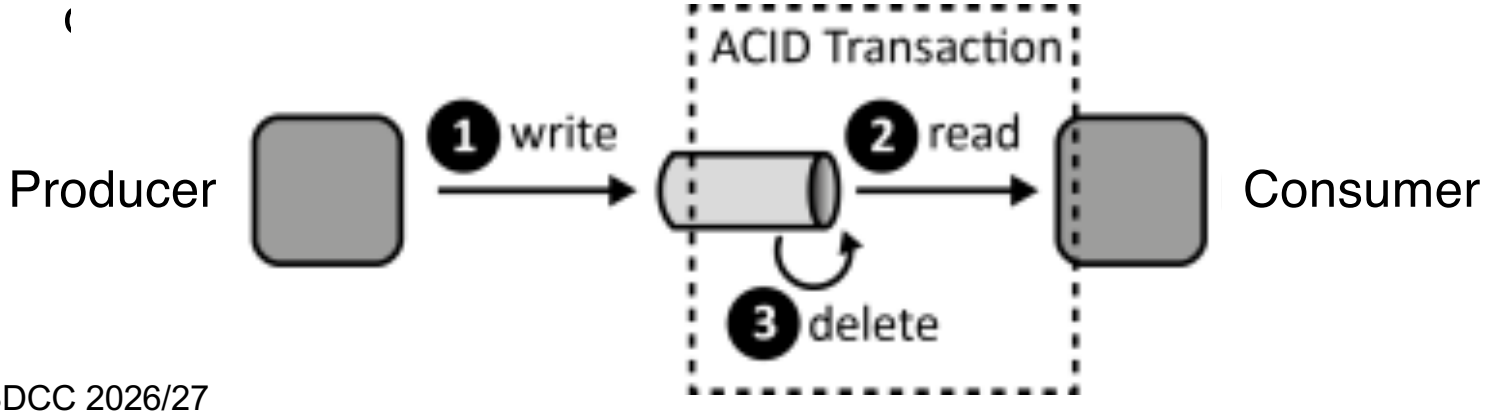


MOM delivery semantics

Transaction-based delivery



- Guarantees messages are deleted from the queue only after they have been successfully received and processed by the consumer
- Mechanism
  - MOM and consumer participate in a transaction: read and delete operations are part of the transaction, ensuring ACID properties
    - Commit: if successful, the message is permanently deleted
    - Rollback: if the transaction fails, the message remains in the

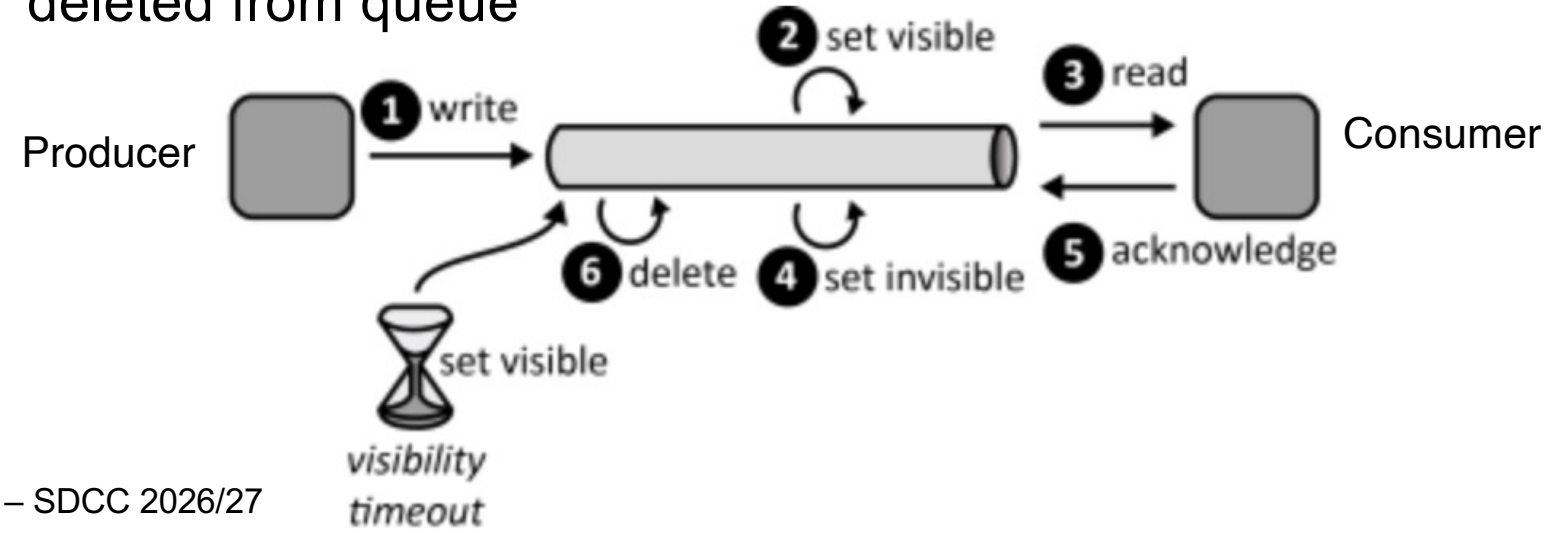


MOM delivery semantics

Timeout-based delivery

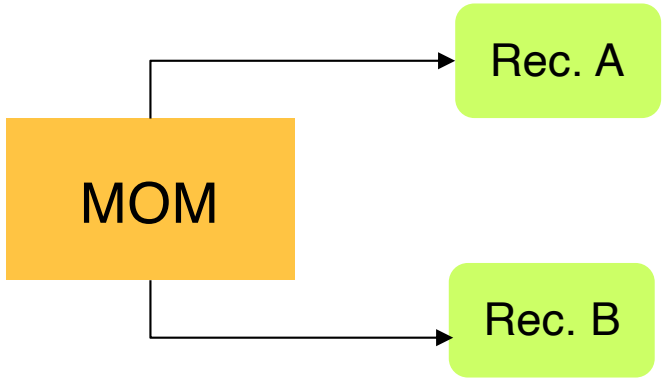


- Ensures messages are delivered at least once while preventing duplicate consumption
- Mechanism
  - Message is not deleted immediately from the queue, but marked as invisible until *visibility timeout* expires
  - Invisible message cannot be read by another consumer
  - After consumer's ack of message processing, message is deleted from queue

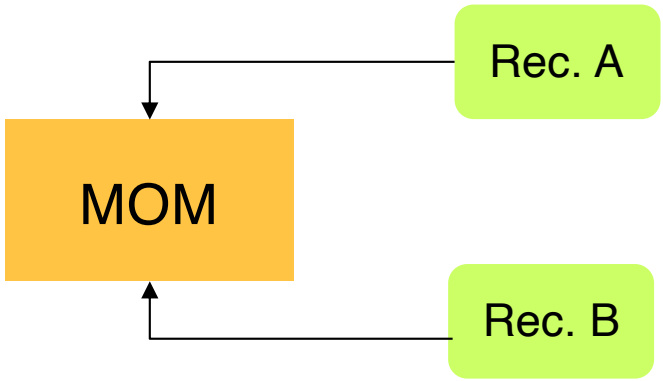


MOM delivery models

- How messages are retrieved by subscribers or consumers
- Options: push vs. pull delivery
- Push: MOM actively notifies the receiver(s) when a message is available
- Pull: receiver(s) periodically polls MOM for new messages



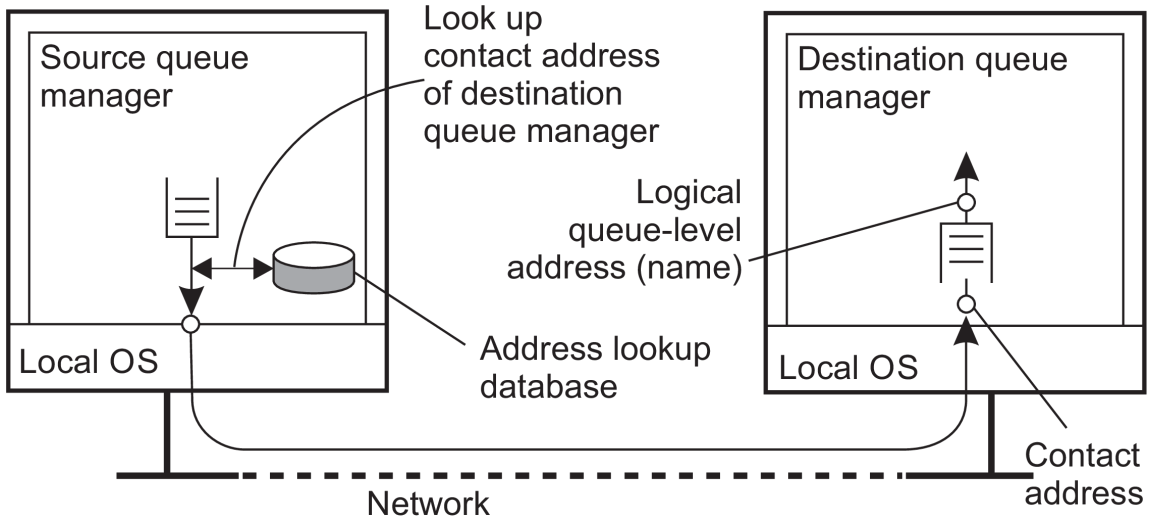
Push delivery



Pull delivery

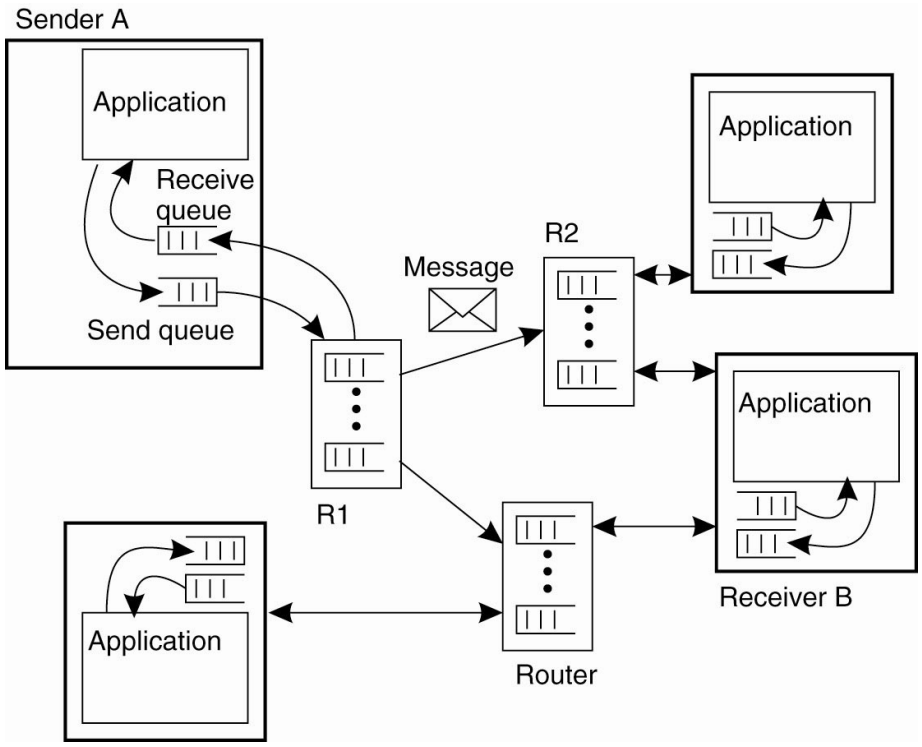
MOM Message routing

- Queues are managed by queue managers (QMs)
  - Applications can only put messages into a local queue
  - Applications can only retrieve a message from a local queue
- QMs are responsible for routing messages between local queues and remote queues
  - Act as message-queuing relays, communicating with distributed applications and other QMs
  - Form an overlay network
  - Some QMs may operate only as routers, without storing messages



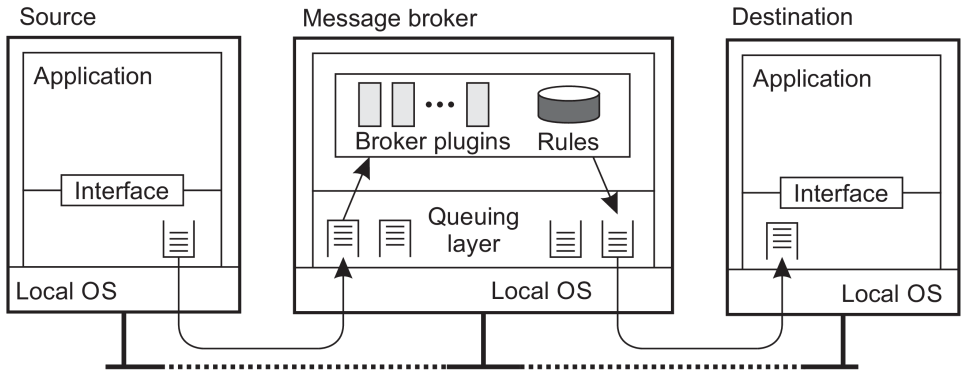
Message routing: overlay network

- Overlay network for message routing
  - Each QM maintains a routing table to map queue names to their location
- Overlay network maintenance
  - Static overlays: routing tables are set up and managed manually
    - Easier to configure but less flexible
  - Dynamic overlays: routing tables are updated at runtime
    - More complex but supports scalable and adaptive routing



Application heterogeneity: message broker

- Problem: new and existing apps rarely share a common data format, how to integrate them into a single, coherent system?
- Solution: ***message broker***
  - Handles application heterogeneity in MOM
- Responsibilities
  - Message transformation: uses a repository of conversion rules to transform message types
  - Content-based routing: routes messages based on content or type
- Scalability and reliability: usually implemented in a distributed manner



MOM frameworks

- Main MOM systems and libraries
  - Apache ActiveMQ https://activemq.apache.org
  - **Apache Kafka**
  - Apache Pulsar https://pulsar.apache.org
  - IBM MQ https://www.ibm.com/products/mq
  - NATS https://nats.io
  - **RabbitMQ**
  - Redis https://redis.io/
  - ZeroMQ https://zeromq.org
- Note: distinction between queue-based and pub/sub patterns is sometimes blurred

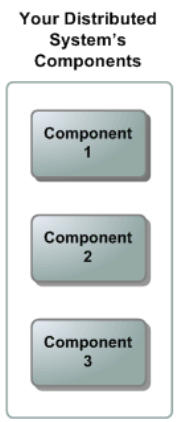
MOM frameworks

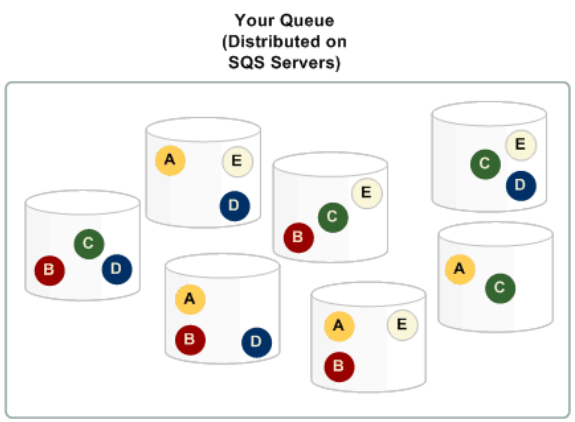
- Cloud-based MOM services
  - **Amazon Simple Queue Service (SQS)**
  - Amazon Simple Notification Service (SNS)
  - CloudAMQP: RabbitMQ as a Service
  - Google Cloud Pub/Sub
  - Microsoft Azure Service Bus
- Note
  - Cloud MOMs abstract infrastructure management, allowing developers to focus on messaging patterns
  - Many support both queue and pub/sub patterns

Amazon Simple Queue Service (SQS)



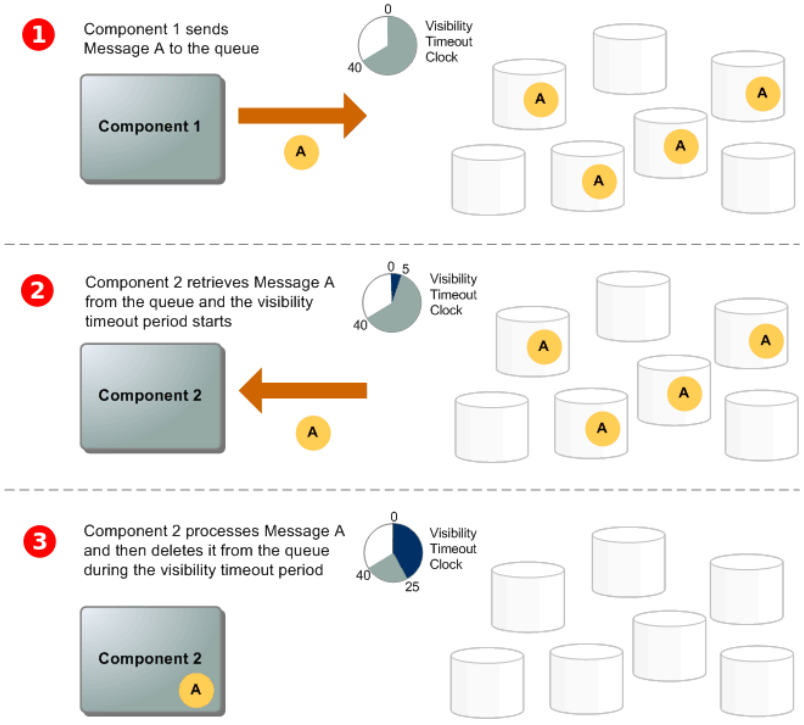
- Reliable, scalable cloud-based message queue service
  - Designed to decouple application components, allowing them to run independently and asynchronously
  - Supports integration across different languages and technologies
- Architecture
  - Message queues fully managed by AWS
  - Queues are distributed across multiple SQS servers
  - Replication within a region: messages stored on multiple servers for high availability
  - Pull delivery model: consumers poll the queue to retrieve messages





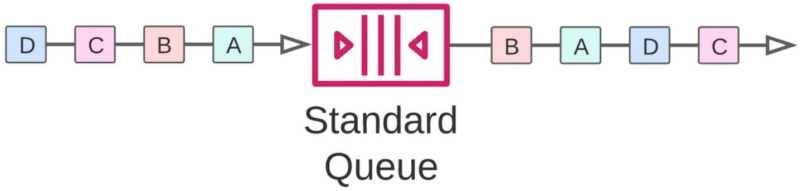
Amazon SQS: Message handling

- Consumer must delete message after processing
  - Queue = temporary holding location, not permanent storage
  - Message retention period: configurable (max 14 days)
- Implements timeout-based delivery
  - When a consumer retrieves a message, it remains in the queue but becomes invisible to others (visibility timeout)
  - If processing succeeds, the consumer deletes the message
  - If processing fails, the timeout expires and the message becomes visible again



Amazon SQS: Polling and queue types

- Consumers poll the queue to receive messages
  - *Short* polling: queries only a subset of SQS servers (may return empty)
  - *Long* polling: queries all servers, wait for messages (reduces empty responses)
- Queue types: standard or FIFO
- Standard queue (default)
  - Best-effort ordering: messages may arrive out-of-order
  - Duplicates possible
- FIFO queue
  - Guarantees in-order delivery (sent=processed order)
  - No duplicates
  - ✗ Lower throughput





Amazon SQS: API

- `CreateQueue`, `ListQueues`, `DeleteQueue`
  - Create a new queue, list all queues, delete an existing queue
- `SendMessage`
  - Add a message to queue (message size up to 256 KB)
  - For larger payloads: store on S3 and send an S3 reference instead
- `ReceiveMessage`
  - Retrieve one or more messages
  - Retrieve up to 10 messages: can specify the maximum number of messages to fetch (from 1 to 10)
  - No filtering before delivery
  - Post-delivery filtering
    - If a consumer wants to filter messages, it must inspect message body or attributes after receiving the messages and decide whether to process or discard them

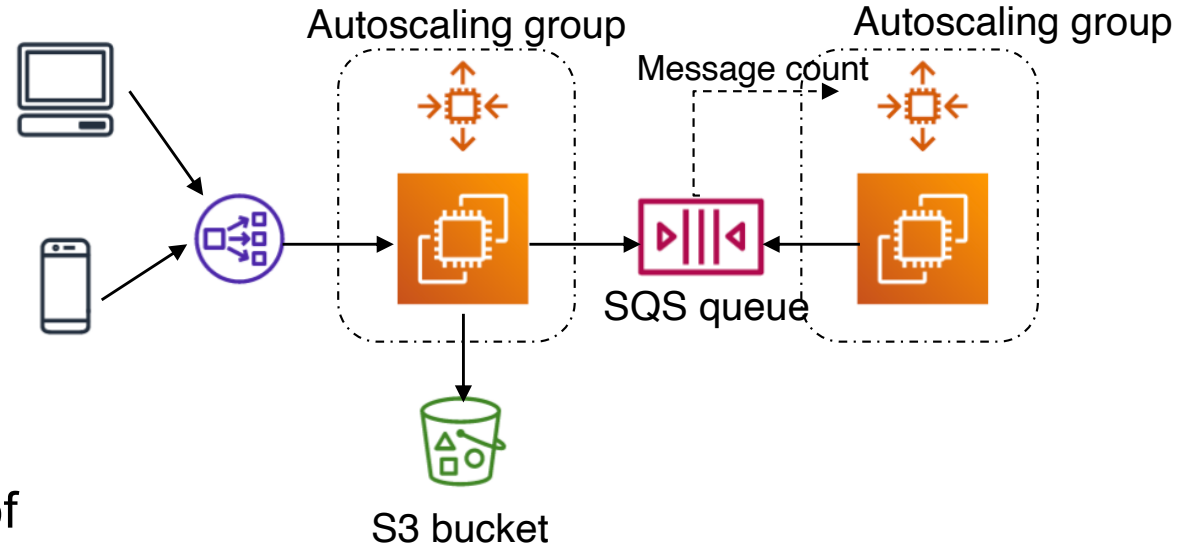
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/APIReference/Welcome.html

Amazon SQS: API

- `DeleteMessage`
  - Remove a specified message from the queue after processing
- `ChangeMessageVisibility`
  - Change the visibility timeout of a specific message in the queue
  - A message remains in the queue until explicitly deleted by the consumer
  - Default visibility timeout: 30 sec.
- `SetQueueAttributes`, `GetQueueAttributes`
  - Modify queue attributes (e.g., message retention), retrieve current queue settings

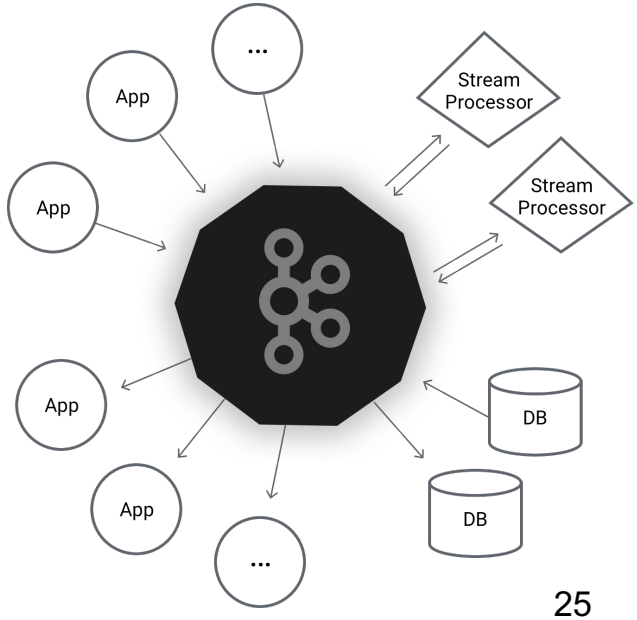
Amazon SQS: example

- Photo processing service using SQS
  - Goal: use SQS to decouple the front-end and back-end components, ensuring load balancing and fault tolerance
  - Front-end uploads a photo to S3 and sends to back-end an SQS message containing the S3 link
  - A pool of EC2 instances (back-end) retrieves messages from the queue
    - Each instances gets a photo, processes it (e.g., resize) and deletes the message after processing
    - If processing fails, the message becomes visible again after the visibility timeout, allowing another instance to retry
    - The back-end pool can be scaled horizontally based on the number of messages in the queue

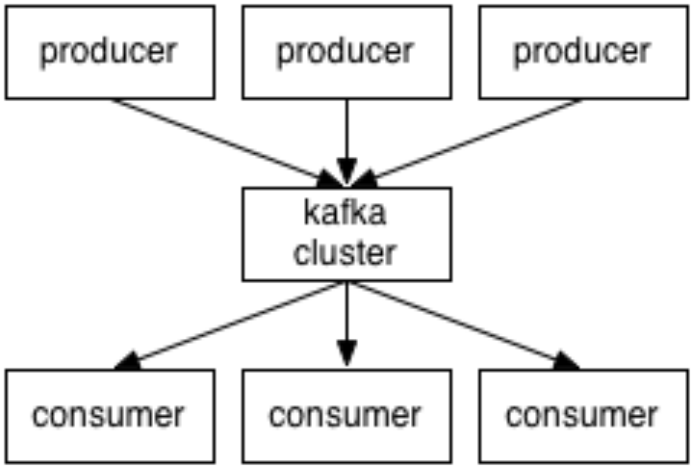


Apache Kafka

- Open-source, distributed event streaming platform https://kafka.apache.org/
- Designed to
  - Publish and subscribe to streams of events
  - Store event streams durably and reliably
  - Process event streams
- Started by LinkedIn in 2010, now Apache project
- Used at scale by LinkedIn, Netflix, Uber, and many others
- Written in Java and Scala
- Horizontally scalable and fault-tolerant
- High-throughput ingestion
  - Billions of events



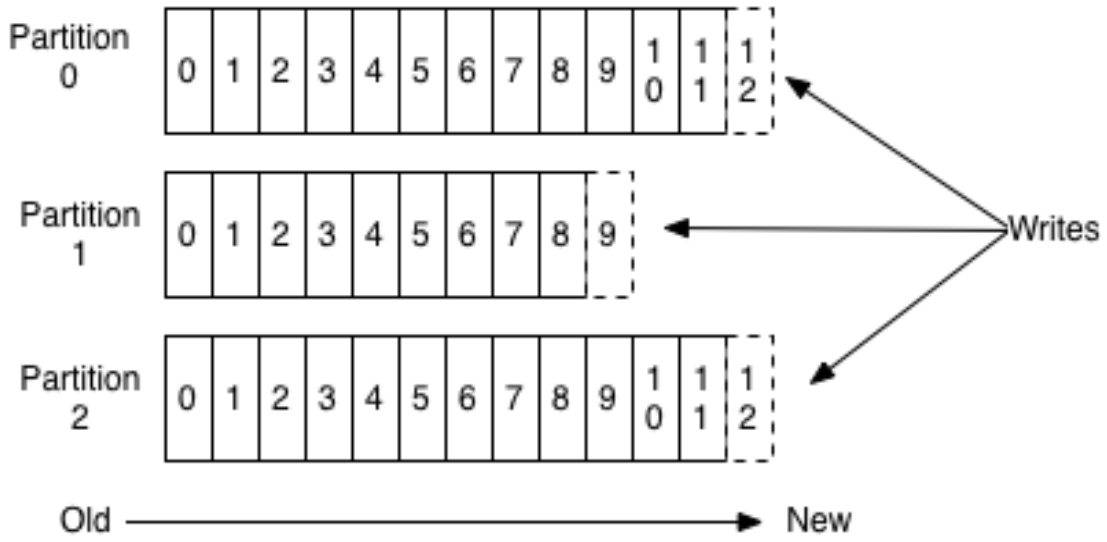
Kafka at a glance



- **Kafka topics**: Kafka stores streams of events (aka *messages* or *records*) in categories known as topics
- **Kafka cluster**: composed of servers called brokers, which can span multiple data centers or cloud regions
  - Brokers receive and store events
- **Producers**: publish (write) events to a Kafka topic
- **Consumers**: subscribe to Kafka topics, read published events and process them
  - A topic can have 0, 1, or many subscribing consumers

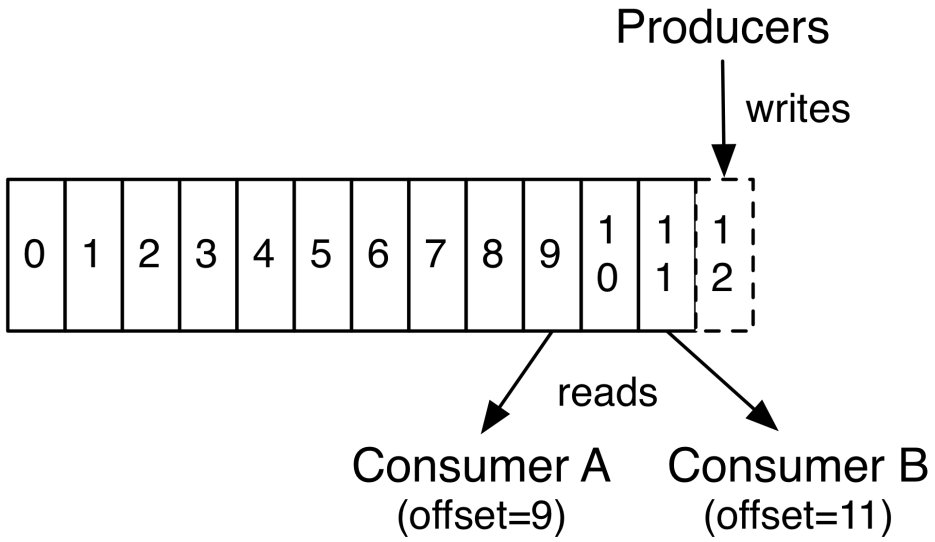
Kafka: topics and partitions

- Topic: category to which events are published
- For each topic, Kafka cluster maintains a **partitioned log**
- Log (data structure): *append-only*, *totally ordered* sequence of events
- Partitioned log: each topic is split into a pre-defined number of partitions
  - Partition: the unit of parallelism for a topic



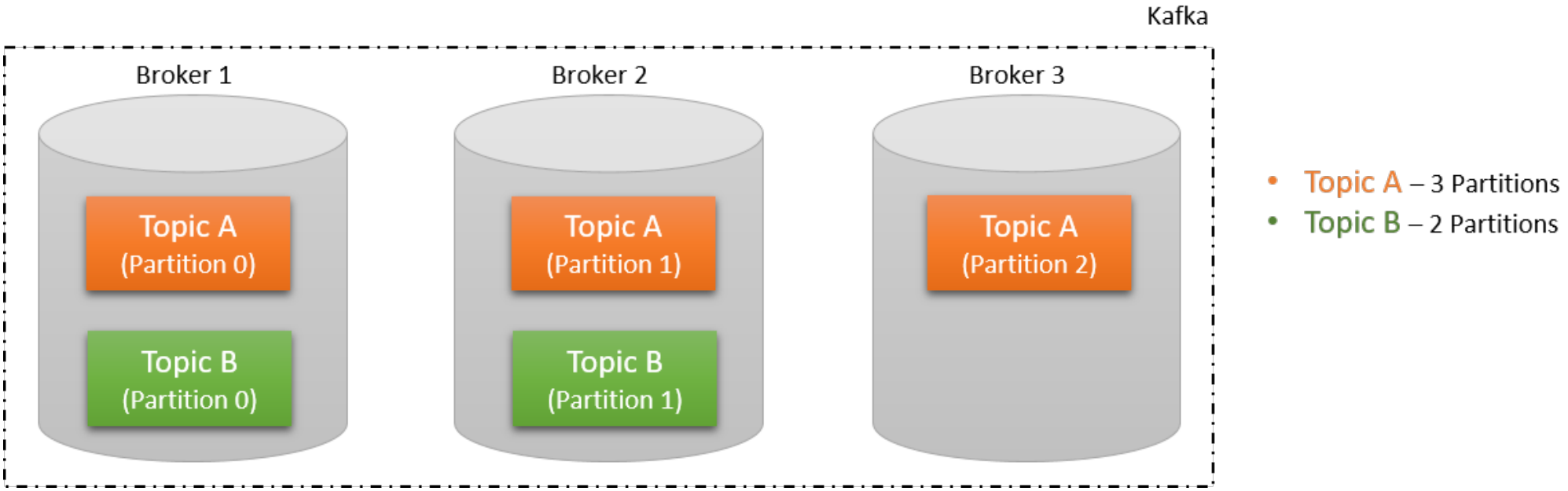
Kafka: partitions and parallelism

- Parallelism and scalability come from having multiple partitions per topic
- Producers publish events to topic partitions
- Consumers read events from the topic
- Each partition is:
  - A numbered, ordered, immutable sequence of records
  - Appended to, never modified *(*immutable records)
  - Similar to a commit log
- Every record has a monotonically increasing **offset**
  - Uniquely identifies each record within a partition



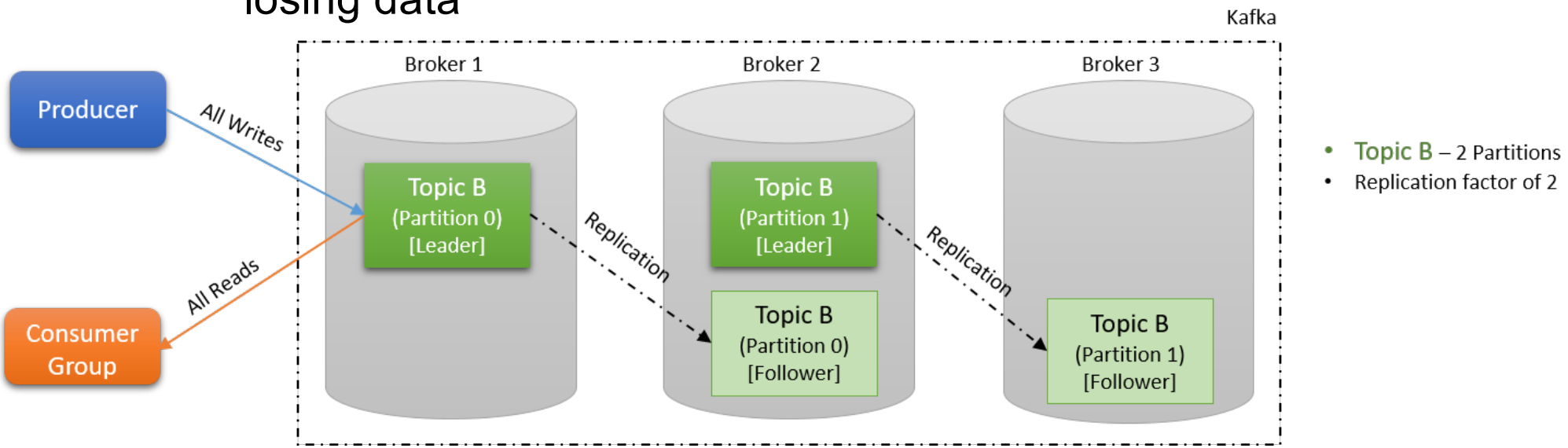
Kafka: partitions and design choices

- Improve scalability: topic partitions are distributed across multiple brokers
  - ✓ I/O throughput increases through parallel reads and writes
    - Multiple producers can write concurrently to different partitions
    - Multiple consumers can read concurrently from different partitions



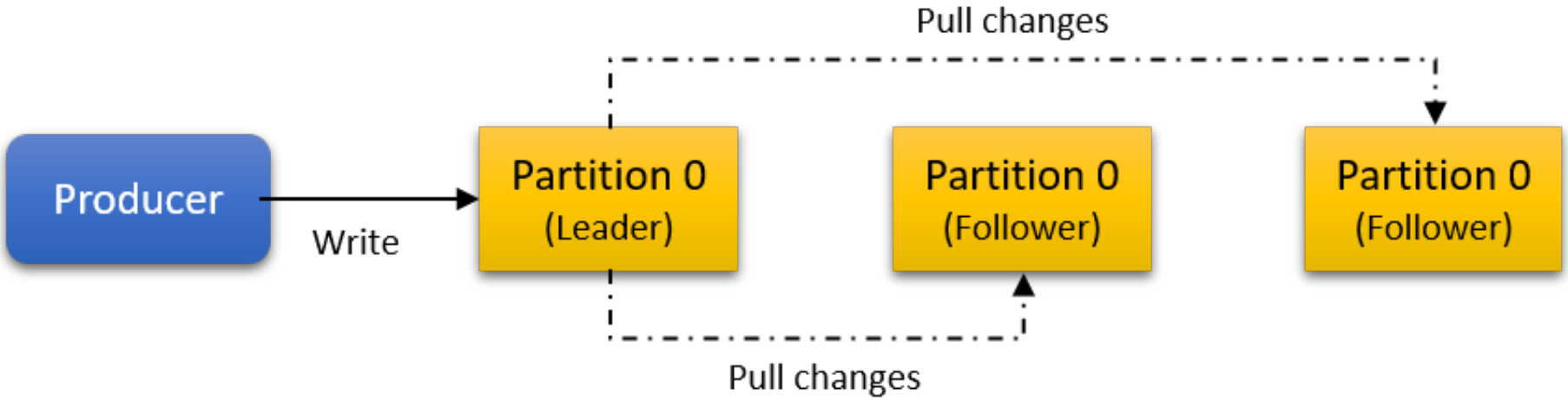
Kafka: partitions and design choices

- Improve fault tolerance: partitions can be *replicated* across brokers
  - Each partition has:
    - 1 leader: handles all writes and reads
    - 0 or more followers: replicate the leader data
- *Replication-factor* = total number of replicas (including leader)
  - Replication-factor = *N* → up to *N*-1 brokers can fail without losing data



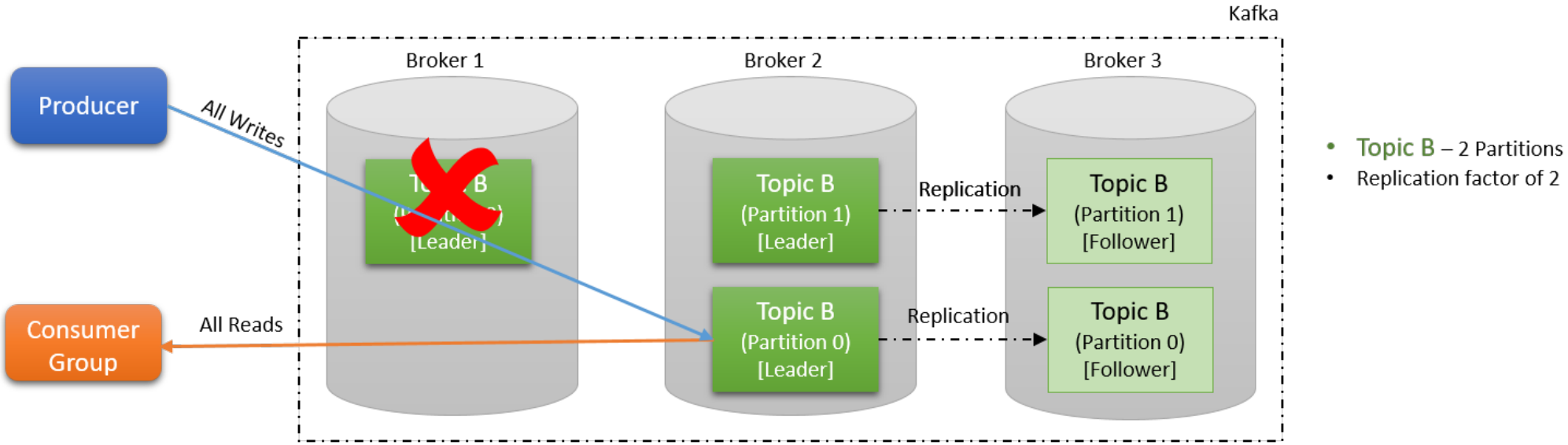
Kafka: partitions and design choices

- Simplify data consistency management: only leader handles all reads and writes
  - Producers write to the leader, consumers read from the leader
  - Followers replicate the leader and serve as backups
  - Followers can be:
    - in-sync: fully updated with the leader
    - out-of-sync: lagging behind the leader



Kafka: partitions and design choices

- Share responsibility and balance load: each broker acts as leader for some partitions and follower for others
- Coordination across brokers is managed by Apache ZooKeeper or KRaft
  - Handles leader election
  - Maintains cluster metadata

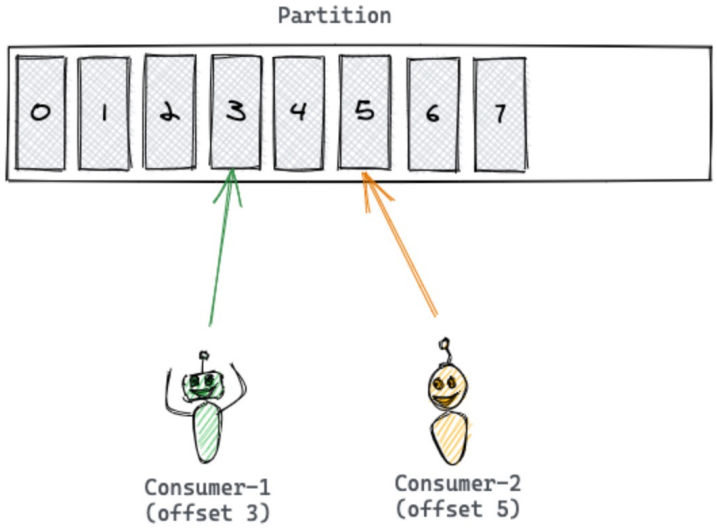


Kafka: producers

- Producers = data sources
- Publish events to topics of their choice
  - Send events directly to the broker that is leader for the target partition (no routing tier)
- Responsible for assigning events to partitions using:
  - Key-based partitioning
    - Event contains a partition key (e.g., order ID, user ID)
    - Partition is chosen by hashing the key
    - Example: if `key=user_id`, all events for the same user go to the same partition
  - Round-robin partitioning
    - Default if no key is specified
  - Custom partitioning
    - A partitioner can be provided to implement a custom partitioning logic
- Multiple producers can write concurrently to the same partition

Kafka: consumers

- Kafka uses a pull-based delivery model for consumers
  https://kafka.apache.org/documentation.html#design_pull
- A consumer tracks its progress using offset
  - Offset identifies which events have been consumed
- Multiple consumers can read the same partition, each reading at different offsets



Kafka: consumers

- Why pull?
- Push model
  - Broker actively pushes events to consumers
  - ✗ Broker must manage
    - Different consumers with their needs and capabilities
    - Transmission rate control
    - Timing decisions: whether to send a message immediately or accumulate more data to send later
- Pull model
  - Consumers control event retrieval from broker
  - Consumers maintain an offset to identify the next event to read
  - ✓ More scalable and flexible
    - Less burden on brokers
    - Consumers can pull events when ready, adjusting the rate
  - ✗ If no events are available, consumers may end up busy waiting for events to arrive

Kafka: consumers

- How can consumers read in a fault-tolerant way?
  - After a consumer reads events, it stores its committed offset in a special Kafka topic called `__consumer_offsets`
  - In case of crash, the consumer can recover by reading events from the committed offset
  - By default, `auto-commit` is enabled

Kafka: message retention and storage

- Kafka brokers store messages reliably on disk
- Messages are stored in log segments (files on disk)
- Unlike other message queue and pub/sub systems, Kafka does not delete messages after delivery, but retains messages based on size or time
- Issue: how to free up disk space?
  - Topics are configured with retention time (how long events should be stored)
  - Upon expiry, events are marked for deletion
  - Alternatively, retention can be specified in bytes
- On each broker, messages are flushed from the broker's in-memory buffer to the disk
  - A log flush policy controls when messages are written to disk (time interval and number of messages)